KATEDRA
INFORMATIKY
UNIVERZITA PALACKÉHO V OLOMOUCI

Paradigmata programování 1

12 Interpret

Michal Krupka

Obsah

1 Implementace vyhodnocovacího procesu

2 Jazyk Scheme

3 Principy interpretu

4 Zásady vytváření interpretu

5 Interpret
- Funkce `ss-eval` a `ss-eval-in-env`
- Vyhodnocování neproměnných atomů
- Prostředí a vyhodnocování symbolů
- Vyhodnocování složeného výrazu a aplikace primitiv
- Speciální operátory `quote` a `if`, makro `let`
- Uživatelské procedury

Vyhodnocovací proces lze naprogramovat

1. zdrojový kód je datová struktura, se kterou umíme pracovat,
2. vyhodnocovací proces je přesně popsaný.

Vyhodnocovací proces lze naprogramovat

1. zdrojový kód je datová struktura, se kterou umíme pracovat,
2. vyhodnocovací proces je přesně popsaný.

funkce `ss-eval`

Obsah

1 Implementace vyhodnocovacího procesu

2 Jazyk Scheme

3 Principy interpretu

4 Zásady vytváření interpretu

5 Interpret
- Funkce `ss-eval` a `ss-eval-in-env`
- Vyhodnocování neproměnných atomů
- Prostředí a vyhodnocování symbolů
- Vyhodnocování složeného výrazu a aplikace primitiv
- Speciální operátory `quote` a `if`, makro `let`
- Uživatelské procedury

Vyhodnocovací proces ve Scheme

Vyhodnocovací proces ve Scheme

- symboly nemají funkční vazby

Vyhodnocovací proces ve Scheme

- symboly nemají funkční vazby
- (funkcím říkáme procedury)

Vyhodnocovací proces ve Scheme

- symboly nemají funkční vazby
- (funkcím říkáme procedury)

Vyhodnocení seznamu

```
(op arg1 arg2 ... argn)
```
Scheme

Vyhodnocovací proces ve Scheme

- symboly nemají funkční vazby
- (funkcím říkáme procedury)

Vyhodnocení seznamu

```
(op arg1 arg2 ... argn)
```
Scheme

1. Jestliže *op* je speciální operátor, celý výraz se vyhodnotí podle jeho pravidel.

Vyhodnocovací proces ve Scheme

- symboly nemají funkční vazby
- (funkcím říkáme procedury)

Vyhodnocení seznamu

```
(op arg1 arg2 ... argn)
```
Scheme

1. Jestliže *op* je speciální operátor, celý výraz se vyhodnotí podle jeho pravidel.
2. Jinak se *op* vyhodnotí. Výsledkem musí být procedura.

Vyhodnocovací proces ve Scheme

- symboly nemají funkční vazby
- (funkcím říkáme procedury)

Vyhodnocení seznamu

```
(op arg1 arg2 ... argn)                                    Scheme
```

1. Jestliže *op* je speciální operátor, celý výraz se vyhodnotí podle jeho pravidel.
2. Jinak se *op* vyhodnotí. Výsledkem musí být procedura.
3. Potom se vyhodnotí argumenty a procedura se na ně aplikuje.

Vyhodnocovací proces ve Scheme

- symboly nemají funkční vazby
- (funkcím říkáme procedury)

Vyhodnocení seznamu

```
(op arg1 arg2 ... argn)
```
Scheme

1. Jestliže *op* je speciální operátor, celý výraz se vyhodnotí podle jeho pravidel.
2. Jinak se *op* vyhodnotí. Výsledkem musí být procedura.
3. Potom se vyhodnotí argumenty a procedura se na ně aplikuje.

op tedy nemusí být symbol:

Vyhodnocovací proces ve Scheme

- symboly nemají funkční vazby
- (funkcím říkáme procedury)

Vyhodnocení seznamu

```
(op arg1 arg2 ... argn)
```
Scheme

1. Jestliže *op* je speciální operátor, celý výraz se vyhodnotí podle jeho pravidel.
2. Jinak se *op* vyhodnotí. Výsledkem musí být procedura.
3. Potom se vyhodnotí argumenty a procedura se na ně aplikuje.

op tedy nemusí být symbol:

```
> ((if (> 1 0) + -) 1 1)
2
```
Scheme

Obsah

1 Implementace vyhodnocovacího procesu

2 Jazyk Scheme

3 Principy interpretu

4 Zásady vytváření interpretu

5 Interpret
- Funkce `ss-eval` a `ss-eval-in-env`
- Vyhodnocování neproměnných atomů
- Prostředí a vyhodnocování symbolů
- Vyhodnocování složeného výrazu a aplikace primitiv
- Speciální operátory `quote` a `if`, makro `let`
- Uživatelské procedury

Interpret a kompilátor

Interpret a kompilátor

Interpret programovacího jazyka: vykonává program krok po kroku bez překladu do jiného jazyka.

Interpret a kompilátor

Interpret programovacího jazyka: vykonává program krok po kroku bez překladu do jiného jazyka.

Kompilátor (*překladač*) programovacího jazyka: převádí program z jednoho jazyka do druhého.

Zadání pro interpret

Zadání pro interpret

- vyhodnocování symbolů a ostatních atomů,
- vyhodnocování složených výrazů,
- prostředí,
- uživatelské funkce jako uzávěry, jejich aplikace,
- vestavěné funkce (*primitiva*) (jako aritmetické operace a funkce na práci se seznamy),
- čtyři základní speciální operátory.

Principy implementace

Principy implementace

1. Při interpretaci programu budeme využívat hodnoty Lispu (páry, symboly, čísla . . .).

Principy implementace

1. Při interpretaci programu budeme využívat hodnoty Lispu (páry, symboly, čísla . . .).
2. Dále budeme využívat primitiva Lispu (např. funkci na sčítání čísel).

Principy implementace

1. Při interpretaci programu budeme využívat hodnoty Lispu (páry, symboly, čísla . . .).
2. Dále budeme využívat primitiva Lispu (např. funkci na sčítání čísel).
3. Program k interpretaci budeme zadávat jako hodnotu jazyka Lisp (atom nebo seznam), nikoliv v textové podobě, jak je obvyklé. Tím odpadne potřeba analyzovat textový zápis programu.

Principy implementace

1. Při interpretaci programu budeme využívat hodnoty Lispu (páry, symboly, čísla . . .).
2. Dále budeme využívat primitiva Lispu (např. funkci na sčítání čísel).
3. Program k interpretaci budeme zadávat jako hodnotu jazyka Lisp (atom nebo seznam), nikoliv v textové podobě, jak je obvyklé. Tím odpadne potřeba analyzovat textový zápis programu.
4. Interpret napíšeme čistě funkcionálně, pouze s použitím znalostí z tohoto semestru.

Principy implementace

1. Při interpretaci programu budeme využívat hodnoty Lispu (páry, symboly, čísla . . .).
2. Dále budeme využívat primitiva Lispu (např. funkci na sčítání čísel).
3. Program k interpretaci budeme zadávat jako hodnotu jazyka Lisp (atom nebo seznam), nikoliv v textové podobě, jak je obvyklé. Tím odpadne potřeba analyzovat textový zápis programu.
4. Interpret napíšeme čistě funkcionálně, pouze s použitím znalostí z tohoto semestru.

Čistě funkcionální interpret

Principy implementace

1. Při interpretaci programu budeme využívat hodnoty Lispu (páry, symboly, čísla . . .).
2. Dále budeme využívat primitiva Lispu (např. funkci na sčítání čísel).
3. Program k interpretaci budeme zadávat jako hodnotu jazyka Lisp (atom nebo seznam), nikoliv v textové podobě, jak je obvyklé. Tím odpadne potřeba analyzovat textový zápis programu.
4. Interpret napíšeme čistě funkcionálně, pouze s použitím znalostí z tohoto semestru.

Čistě funkcionální interpret

- prostředí nelze modifikovat (neexistuje operátor `bind`)

Principy implementace

1. Při interpretaci programu budeme využívat hodnoty Lispu (páry, symboly, čísla . . .).
2. Dále budeme využívat primitiva Lispu (např. funkci na sčítání čísel).
3. Program k interpretaci budeme zadávat jako hodnotu jazyka Lisp (atom nebo seznam), nikoliv v textové podobě, jak je obvyklé. Tím odpadne potřeba analyzovat textový zápis programu.
4. Interpret napíšeme čistě funkcionálně, pouze s použitím znalostí z tohoto semestru.

Čistě funkcionální interpret

- prostředí nelze modifikovat (neexistuje operátor `bind`)
- jak napsat rekurzi?

Funkce `ss-eval`: příklady (základ)

Funkce `ss-eval`: příklady (základ)

```
> (ss-eval 1)
1
```

Funkce `ss-eval`: příklady (základ)

```
> (ss-eval 1)
1

> (ss-eval 'pi)
3.141592653589793D0
```

Funkce `ss-eval`: příklady (základ)

```
> (ss-eval 1)
1

> (ss-eval 'pi)
3.141592653589793D0

> (ss-eval '(quote a))
A
```

Funkce `ss-eval`: příklady (základ)

```
> (ss-eval 1)
1

> (ss-eval 'pi)
3.141592653589793D0

> (ss-eval '(quote a))
A

> (ss-eval '(+ 1 2))
3
```

Funkce `ss-eval`: příklady (základ)

```
> (ss-eval 1)
1

> (ss-eval 'pi)
3.141592653589793D0

> (ss-eval '(quote a))
A

> (ss-eval '(+ 1 2))
3

> (ss-eval '((if (> 1 0) + -) 1 1))
2
```

Funkce `ss-eval`: příklady (vazby a procedury)

Funkce `ss-eval`: příklady (vazby a procedury)

```
> (ss-eval '(let ((x 1)
                  (y 2))
              (+ x y)))
3
```

Funkce `ss-eval`: příklady (vazby a procedury)

```
> (ss-eval '(let ((x 1)
                  (y 2))
              (+ x y)))
3

> (ss-eval '(let ((x 1)
                  (y 2))
              ((lambda (x) (* x y)) 5)))
10
```

Funkce `ss-eval`: příklady (vazby a procedury)

```
> (ss-eval '(let ((x 1)
                  (y 2))
              (+ x y)))
3

> (ss-eval '(let ((x 1)
                  (y 2))
              ((lambda (x) (* x y)) 5)))
10

> (ss-eval '(let ((avg (lambda (x y)
                         (/ (+ x y) 2))))
              (avg 1 3)))
2
```

Funkce `ss-eval`: příklady (rekurze)

Funkce `ss-eval`: příklady (rekurze)

```
> (ss-eval '(let ((fact (lambda (n)
                          (let ((fct (lambda (f n)
                                       (if (= n 0)
                                           1
                                         (* n (f f (- n 1)))))))
                            (fct fct n)))))
              (fact 5)))
120
```

Funkce `ss-eval-in-env`: příklady

Funkce `ss-eval-in-env`: příklady

Předpokládejme, že v prostředí `env` je `x` navázáno na `1` a `y` na `2`.

Funkce `ss-eval-in-env`: příklady

Předpokládejme, že v prostředí `env` je `x` navázáno na `1` a `y` na `2`.

```
> (ss-eval-in-env '(+ x y) env)
3
```

Funkce `ss-eval-in-env`: příklady

Předpokládejme, že v prostředí `env` je `x` navázáno na `1` a `y` na `2`.

```
> (ss-eval-in-env '(+ x y) env)
3

> (ss-eval-in-env '((lambda (x) (* x y)) 5) env)
10
```

Funkce `ss-eval-in-env`: příklady

Předpokládejme, že v prostředí `env` je `x` navázáno na `1` a `y` na `2`.

```
> (ss-eval-in-env '(+ x y) env)
3

> (ss-eval-in-env '((lambda (x) (* x y)) 5) env)
10
```

Funkci napíšeme přesně podle pravidel vyhodnocovacího procesu Scheme.

Obsah

1 Implementace vyhodnocovacího procesu

2 Jazyk Scheme

3 Principy interpretu

4 Zásady vytváření interpretu

5 Interpret
- Funkce `ss-eval` a `ss-eval-in-env`
- Vyhodnocování neproměnných atomů
- Prostředí a vyhodnocování symbolů
- Vyhodnocování složeného výrazu a aplikace primitiv
- Speciální operátory `quote` a `if`, makro `let`
- Uživatelské procedury

Způsob práce

1. Dělit složitější problémy na jednodušší.

Způsob práce

1. Dělit složitější problémy na jednodušší.
2. Stále udržovat funkční program.

Obsah

1 Implementace vyhodnocovacího procesu

2 Jazyk Scheme

3 Principy interpretu

4 Zásady vytváření interpretu

5 Interpret
- Funkce `ss-eval` a `ss-eval-in-env`
- Vyhodnocování neproměnných atomů
- Prostředí a vyhodnocování symbolů
- Vyhodnocování složeného výrazu a aplikace primitiv
- Speciální operátory `quote` a `if`, makro `let`
- Uživatelské procedury

Funkce `ss-eval` a `ss-eval-in-env`

Funkce ss-eval a ss-eval-in-env

```
(defun ss-eval (expr)
  (ss-eval-in-env expr (ss-global-environment)))
```

Funkce ss-eval a ss-eval-in-env

```
(defun ss-eval (expr)
  (ss-eval-in-env expr (ss-global-environment)))
```

Funkce ss-eval-in-env

Funkce ss-eval a ss-eval-in-env

```
(defun ss-eval (expr)
  (ss-eval-in-env expr (ss-global-environment)))
```

Funkce ss-eval-in-env

```
(defun ss-eval-in-env (expr env)
```

Funkce ss-eval a ss-eval-in-env

```
(defun ss-eval (expr)
  (ss-eval-in-env expr (ss-global-environment)))
```

Funkce ss-eval-in-env

```
(defun ss-eval-in-env (expr env)
  (if (ss-compound-expr-p expr)
```

Funkce ss-eval a ss-eval-in-env

```
(defun ss-eval (expr)
  (ss-eval-in-env expr (ss-global-environment)))
```

Funkce ss-eval-in-env

```
(defun ss-eval-in-env (expr env)
  (if (ss-compound-expr-p expr)
      (ss-eval-compound-expr expr env)
```

Funkce ss-eval a ss-eval-in-env

```
(defun ss-eval (expr)
  (ss-eval-in-env expr (ss-global-environment)))
```

Funkce ss-eval-in-env

```
(defun ss-eval-in-env (expr env)
  (if (ss-compound-expr-p expr)
      (ss-eval-compound-expr expr env)
    (ss-eval-atom expr env)))
```

Funkce ss-eval a ss-eval-in-env

```
(defun ss-eval (expr)
  (ss-eval-in-env expr (ss-global-environment)))
```

Funkce ss-eval-in-env

```
(defun ss-eval-in-env (expr env)
  (if (ss-compound-expr-p expr)
      (ss-eval-compound-expr expr env)
    (ss-eval-atom expr env)))
```

Je daný výraz složený?

Funkce ss-eval a ss-eval-in-env

```
(defun ss-eval (expr)
  (ss-eval-in-env expr (ss-global-environment)))
```

Funkce ss-eval-in-env

```
(defun ss-eval-in-env (expr env)
  (if (ss-compound-expr-p expr)
      (ss-eval-compound-expr expr env)
    (ss-eval-atom expr env)))
```

Je daný výraz složený?

```
(defun ss-compound-expr-p (expr)
  (consp expr))
```

Vyhodnocení atomu

Vyhodnocení atomu

```
(defun ss-eval-atom (atom env)
```

Vyhodnocení atomu

```
(defun ss-eval-atom (atom env)
  (if (symbolp atom)
```

Vyhodnocení atomu

```
(defun ss-eval-atom (atom env)
  (if (symbolp atom)
      (ss-symbol-value atom env)
```

Vyhodnocení atomu

```
(defun ss-eval-atom (atom env)
  (if (symbolp atom)
      (ss-symbol-value atom env)
    atom))
```

Vyhodnocení atomu

```
(defun ss-eval-atom (atom env)
  (if (symbolp atom)
      (ss-symbol-value atom env)
    atom))
```

Test:

Vyhodnocení atomu

```
(defun ss-eval-atom (atom env)
  (if (symbolp atom)
      (ss-symbol-value atom env)
    atom))
```

Test:

```
> (ss-eval 1)
1
```

Reprezentace prostředí

Prostředí reprezentujeme datovou strukturou.

Reprezentace prostředí

Prostředí reprezentujeme datovou strukturou.

Víme, že prostředí musí obsahovat:

Reprezentace prostředí

Prostředí reprezentujeme datovou strukturou.

Víme, že prostředí musí obsahovat:

1. tabulku vazeb symbolů,

Reprezentace prostředí

Prostředí reprezentujeme datovou strukturou.

Víme, že prostředí musí obsahovat:

1. tabulku vazeb symbolů,
2. svého předka.

Reprezentace prostředí

Prostředí reprezentujeme datovou strukturou.

Víme, že prostředí musí obsahovat:

1. tabulku vazeb symbolů,
2. svého předka.

Prostředí

```
(ss-env symbols values parent)
```

symbols: seznam symbolů
values: seznam hodnot
parent: předek prostředí (jiné prostředí) nebo `nil`

(nemusíme rozlišovat typ vazby)

Konstruktor prostředí

Konstruktor prostředí

```
(defun ss-make-env (symbols values parent)
  (list 'ss-env symbols values parent))
```

Konstruktor prostředí

```
(defun ss-make-env (symbols values parent)
  (list 'ss-env symbols values parent))
```

Testy (druhý použijeme dále):

Konstruktor prostředí

```
(defun ss-make-env (symbols values parent)
  (list 'ss-env symbols values parent))
```

Testy (druhý použijeme dále):

```
> (ss-make-env '(a b) '(0 1) nil)
(SS-ENV (A B) (0 1) NIL)
```

Konstruktor prostředí

```
(defun ss-make-env (symbols values parent)
  (list 'ss-env symbols values parent))
```

Testy (druhý použijeme dále):

```
> (ss-make-env '(a b) '(0 1) nil)
(SS-ENV (A B) (0 1) NIL)
```

```
> (ss-make-env '(a b)
               '(0 1)
               (ss-make-env '(x y) '(u v) nil)))
(SS-ENV (A B) (0 1) (SS-ENV (X Y) (U V) NIL))
```

Selektory prostředí

Selektory prostředí

```
(defun ss-env-symbols (env)
  (second env))
```

Selektory prostředí

```
(defun ss-env-symbols (env)
  (second env))

(defun ss-env-values (env)
  (third env))
```

Selektory prostředí

```
(defun ss-env-symbols (env)
  (second env))

(defun ss-env-values (env)
  (third env))

(defun ss-env-parent (env)
  (fourth env))
```

Funkce `ss-symbol-value`

Funkce `ss-symbol-value`

má zjistit hodnotu vazby symbolu v daném prostředí. To obnáší:

Funkce `ss-symbol-value`

má zjistit hodnotu vazby symbolu v daném prostředí. To obnáší:

1. zjistit, zda má symbol vazbu v daném prostředí,

Funkce `ss-symbol-value`

má zjistit hodnotu vazby symbolu v daném prostředí. To obnáší:

1. zjistit, zda má symbol vazbu v daném prostředí,
2. pokud nemá, hledat vazbu rekurzivně v předcích prostředí,

Funkce `ss-symbol-value`

má zjistit hodnotu vazby symbolu v daném prostředí. To obnáší:

1. zjistit, zda má symbol vazbu v daném prostředí,
2. pokud nemá, hledat vazbu rekurzivně v předcích prostředí,
3. vrátit hodnotu nalezené vazby.

Funkce `ss-symbol-value`

Funkce ss-symbol-value

```
(defun ss-symbol-value (symbol env)
  (let ((index (position symbol (ss-env-symbols env))))
    (if (null index)
        (ss-symbol-value symbol (ss-env-parent env))
      (nth index (ss-env-values env)))))
```

Testy

Testy

```
> (bind env (ss-make-env '(a b)
                         '(0 1)
                         (ss-make-env '(x y) '(u v) nil)))
(SS-ENV (A B) (0 1) (SS-ENV (X Y) (U V) NIL))
```

Testy

```
> (bind env (ss-make-env '(a b)
                         '(0 1)
                         (ss-make-env '(x y) '(u v) nil)))
(SS-ENV (A B) (0 1) (SS-ENV (X Y) (U V) NIL))

> (ss-eval-in-env 'a env)
0
```

Testy

```
> (bind env (ss-make-env '(a b)
                         '(0 1)
                         (ss-make-env '(x y) '(u v) nil)))
(SS-ENV (A B) (0 1) (SS-ENV (X Y) (U V) NIL))

> (ss-eval-in-env 'a env)
0

> (ss-eval-in-env 'y env)
V
```

Globální prostředí

Globální prostředí

Funkce `ss-global-environment`:

Globální prostředí

Funkce `ss-global-environment`:

```
(defun ss-global-environment ()
  (ss-make-env '( zero pi + - * / = cons)
               (list 0 pi #'+ #'- #'* #'/ #'= #'cons)
               nil))
```

Globální prostředí

Funkce `ss-global-environment`:

```
(defun ss-global-environment ()
  (ss-make-env '( zero pi + - * / = cons)
               (list 0 pi #'+ #'- #'* #'/ #'= #'cons)
               nil))
```

Test:

Globální prostředí

Funkce `ss-global-environment`:

```
(defun ss-global-environment ()
  (ss-make-env '( zero pi + - * / = cons)
               (list 0 pi #'+ #'- #'* #'/ #'= #'cons)
               nil))
```

Test:

```
> (ss-symbol-value 'zero (ss-global-environment))
0
```

Test vyhodnocování proměnných

Test vyhodnocování proměnných

```
> (ss-eval 'zero)
0
```

Test vyhodnocování proměnných

```
> (ss-eval 'zero)
0

> (ss-eval '=)
#<FUNCTION =>
```

Vyhodnocení složeného výrazu

Vyhodnocení složeného výrazu

Funkce `ss-eval-compound-expr`

Vyhodnocení složeného výrazu

Funkce `ss-eval-compound-expr`

- výrazy s operátory `quote`, `if` a `lambda` vyhodnotí podle jejich pravidel,

Vyhodnocení složeného výrazu

Funkce `ss-eval-compound-expr`

- výrazy s operátory `quote`, `if` a `lambda` vyhodnotí podle jejich pravidel,
- `let`-výraz převede na jiný výraz a ten vyhodnotí (jako *makro*),

Vyhodnocení složeného výrazu

Funkce `ss-eval-compound-expr`

- výrazy s operátory `quote`, `if` a `lambda` vyhodnotí podle jejich pravidel,
- `let`-výraz převede na jiný výraz a ten vyhodnotí (jako *makro*),
- ostatní vyhodnotí jako aplikaci procedury.

Vyhodnocení složeného výrazu

Vyhodnocení složeného výrazu

```
(defun ss-eval-compound-expr (e env)
  (let ((op (car e))
        (args (cdr e)))
```

Vyhodnocení složeného výrazu

```
(defun ss-eval-compound-expr (e env)
  (let ((op (car e))
        (args (cdr e)))
    (cond ((eql op 'quote)
           (apply #'ss-eval-quote-expr args))
```

Vyhodnocení složeného výrazu

```
(defun ss-eval-compound-expr (e env)
  (let ((op (car e))
        (args (cdr e)))
    (cond ((eql op 'quote)
           (apply #'ss-eval-quote-expr args))
          ((eql op 'if)
           (apply #'ss-eval-if-expr env args))
```

Vyhodnocení složeného výrazu

```
(defun ss-eval-compound-expr (e env)
  (let ((op (car e))
        (args (cdr e)))
    (cond ((eql op 'quote)
           (apply #'ss-eval-quote-expr args))
          ((eql op 'if)
           (apply #'ss-eval-if-expr env args))
          ((eql op 'lambda)
           (apply #'ss-eval-lambda-expr env args))
```

Vyhodnocení složeného výrazu

```
(defun ss-eval-compound-expr (e env)
  (let ((op (car e))
        (args (cdr e)))
    (cond ((eql op 'quote)
           (apply #'ss-eval-quote-expr args))
          ((eql op 'if)
           (apply #'ss-eval-if-expr env args))
          ((eql op 'lambda)
           (apply #'ss-eval-lambda-expr env args))
          ((eql op 'let)
           (ss-eval-in-env (ss-expand-let-expr (first args)
                                               (second args))
                           env))
```

Vyhodnocení složeného výrazu

```
(defun ss-eval-compound-expr (e env)
  (let ((op (car e))
        (args (cdr e)))
    (cond ((eql op 'quote)
           (apply #'ss-eval-quote-expr args))
          ((eql op 'if)
           (apply #'ss-eval-if-expr env args))
          ((eql op 'lambda)
           (apply #'ss-eval-lambda-expr env args))
          ((eql op 'let)
           (ss-eval-in-env (ss-expand-let-expr (first args)
                                               (second args))
                           env))
          (t (ss-eval-application env e)))))
```

Vyhodnocení aplikace

Vyhodnocení aplikace

```
(defun ss-eval-application (env expr)
  (let ((vals (ss-eval-exprs expr env)))
    (ss-apply-proc (car vals) (cdr vals))))
```

Vyhodnocení aplikace

```
(defun ss-eval-application (env expr)
  (let ((vals (ss-eval-exprs expr env)))
    (ss-apply-proc (car vals) (cdr vals))))
```

Pomocná funkce:

Vyhodnocení aplikace

```
(defun ss-eval-application (env expr)
  (let ((vals (ss-eval-exprs expr env)))
    (ss-apply-proc (car vals) (cdr vals))))
```

Pomocná funkce:

```
(defun ss-eval-exprs (list env)
  (mapcar (lambda (expr)
            (ss-eval-in-env expr env))
          list))
```

(doplníme na začátek souboru)

Vyhodnocení aplikace

Vyhodnocení aplikace

Aplikace procedury:

Vyhodnocení aplikace

Aplikace procedury:

```
(defun ss-apply-proc (proc args)
```

Vyhodnocení aplikace

Aplikace procedury:

```
(defun ss-apply-proc (proc args)
  (if (ss-primitive-p proc)
```

Vyhodnocení aplikace

Aplikace procedury:

```
(defun ss-apply-proc (proc args)
  (if (ss-primitive-p proc)
      (ss-apply-primitive proc args)
```

Vyhodnocení aplikace

Aplikace procedury:

```
(defun ss-apply-proc (proc args)
  (if (ss-primitive-p proc)
      (ss-apply-primitive proc args)
    (ss-apply-user-proc proc args)))
```

Vyhodnocení aplikace

Aplikace procedury:

```
(defun ss-apply-proc (proc args)
  (if (ss-primitive-p proc)
      (ss-apply-primitive proc args)
    (ss-apply-user-proc proc args)))
```

Co je primitivum:

Vyhodnocení aplikace

Aplikace procedury:

```
(defun ss-apply-proc (proc args)
  (if (ss-primitive-p proc)
      (ss-apply-primitive proc args)
    (ss-apply-user-proc proc args)))
```

Co je primitivum:

```
(defun ss-primitive-p (proc)
  (functionp proc))
```

Aplikace primitiva

Aplikace primitiva

```
(defun ss-apply-primitive (proc args)
```

Aplikace primitiva

```
(defun ss-apply-primitive (proc args)
  (apply proc args))
```

Aplikace primitiva

```
(defun ss-apply-primitive (proc args)
  (apply proc args))
```

Test:

Aplikace primitiva

```
(defun ss-apply-primitive (proc args)
  (apply proc args))
```

Test:

```
> (ss-eval '(+ 1 2))
3

> (ss-eval '(cons (+ 1 2) (* 3 4)))
(3 . 12)
```

Aplikace uživatelské procedury bude později.

quote

quote

```
(defun ss-eval-quote-expr (arg)
```

quote

```
(defun ss-eval-quote-expr (arg)
  arg)
```

quote

```
(defun ss-eval-quote-expr (arg)
  arg)
```

Test:

quote

```
(defun ss-eval-quote-expr (arg)
  arg)
```

Test:

```
> (ss-eval '(quote a))
A

> (ss-eval '(quote (a b c)))
(A B C)

> (ss-eval ''x)
```

quote

```
(defun ss-eval-quote-expr (arg)
  arg)
```

Test:

```
> (ss-eval '(quote a))
A

> (ss-eval '(quote (a b c)))
(A B C)

> (ss-eval ''x)
X
```

if

if

```
(defun ss-eval-if-expr (env test then else)
```

if

```
(defun ss-eval-if-expr (env test then else)
  (if (ss-eval-in-env test env)
```

if

```
(defun ss-eval-if-expr (env test then else)
  (if (ss-eval-in-env test env)
      (ss-eval-in-env then env)
```

if

```
(defun ss-eval-if-expr (env test then else)
  (if (ss-eval-in-env test env)
      (ss-eval-in-env then env)
    (ss-eval-in-env else env)))
```

if

```
(defun ss-eval-if-expr (env test then else)
  (if (ss-eval-in-env test env)
      (ss-eval-in-env then env)
    (ss-eval-in-env else env)))
```

Test:

if

```
(defun ss-eval-if-expr (env test then else)
  (if (ss-eval-in-env test env)
      (ss-eval-in-env then env)
    (ss-eval-in-env else env)))
```

Test:

```
> (ss-eval '(if 't 1 2))
1

> (ss-eval '(if 'nil 1 2))
2

> (ss-eval '(if (= 1 2) (+ 1 2) (* 3 4)))
12
```

let

`let`

Definujeme jako makro:

`let`

Definujeme jako makro:

1. `let`-výraz převedeme na jiný výraz,

`let`

Definujeme jako makro:

1. `let`-výraz převedeme na jiný výraz,
2. ten vyhodnotíme.

(druhé jsme už udělali ve funkci `ss-eval-compound-expr`)

Převod (expanze) `let`-výrazu

Převod (expanze) `let`-výrazu

```
(let ((a 1)                ((lambda (a b) (+ a b))
      (b 2))      ->        1
  (+ a b))                  2)
```

Převod (expanze) `let`-výrazu

```
(let ((a 1)                ((lambda (a b) (+ a b))
      (b 2))      ->        1
  (+ a b))                  2)
```

```
(defun ss-expand-let-expr (bindings body)
  (cons (list 'lambda
              (mapcar #'first bindings)
              body)
        (mapcar #'second bindings)))
```

Reprezentace uživatelských procedur

Reprezentace uživatelských procedur

Jak víme, uživatelské procedury obsahují tři údaje:

Reprezentace uživatelských procedur

Jak víme, uživatelské procedury obsahují tři údaje:

1. seznam parametrů,

Reprezentace uživatelských procedur

Jak víme, uživatelské procedury obsahují tři údaje:

1. seznam parametrů,
2. tělo,

Reprezentace uživatelských procedur

Jak víme, uživatelské procedury obsahují tři údaje:

1. seznam parametrů,
2. tělo,
3. prostředí vzniku.

Reprezentace uživatelských procedur

Jak víme, uživatelské procedury obsahují tři údaje:

1. seznam parametrů,
2. tělo,
3. prostředí vzniku.

Reprezentace:

Reprezentace uživatelských procedur

Jak víme, uživatelské procedury obsahují tři údaje:

1. seznam parametrů,
2. tělo,
3. prostředí vzniku.

Reprezentace:

```
(ss-proc params body env)
```

Konstruktor a selektory

Konstruktor a selektory

Struktura procedury:

Konstruktor a selektory

Struktura procedury:

```
(ss-proc params body env)
```

Konstruktor a selektory

Struktura procedury:

```
(ss-proc params body env)
```

Konstruktor a selektory:

Konstruktor a selektory

Struktura procedury:

```
(ss-proc params body env)
```

Konstruktor a selektory:

```
(defun ss-make-proc (params body env)
  (list 'ss-proc params body env))
```

Konstruktor a selektory

Struktura procedury:

```
(ss-proc params body env)
```

Konstruktor a selektory:

```
(defun ss-make-proc (params body env)
  (list 'ss-proc params body env))

(defun ss-proc-params (proc)
  (second proc))

(defun ss-proc-body (proc)
  (third proc))

(defun ss-proc-env (proc)
  (fourth proc))
```

Speciální operátor `lambda`

Speciální operátor `lambda`

```
(defun ss-eval-lambda-expr (env params body)
```

Speciální operátor `lambda`

```
(defun ss-eval-lambda-expr (env params body)
  (ss-make-proc params body env))
```

Aplikace uživatelské procedury

Aplikace uživatelské procedury

```
(defun ss-apply-user-proc (proc args)
  (ss-eval-in-env
```

Aplikace uživatelské procedury

```
(defun ss-apply-user-proc (proc args)
  (ss-eval-in-env (ss-proc-body proc)
```

Aplikace uživatelské procedury

```
(defun ss-apply-user-proc (proc args)
  (ss-eval-in-env (ss-proc-body proc)
                  (ss-make-env (ss-proc-params proc)
                               args
                               (ss-proc-env proc))))
```